Assignment-13.2

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

• Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

• Ask AI to locate repeated computations within the program.

• Encapsulate repeated logic into reusable functions.

• Verify that program output remains unchanged after refactoring.

• Add proper docstrings to all defined functions.

Sample Code:

```
length = 8

width = 4

print("Area:", length * width)

print("Perimeter:", 2 * (length + width))

length = 12

width = 6

print("Area:", length * width)

print("Perimeter:", 2 * (length + width))
```

Expected Output:

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

Code:

```python
def calculate_rectangle(length, width):
    area = length * width
    perimeter = 2 * (length + width)
    print("Area:", area)
    print("Perimeter:", perimeter)

# First rectangle
length = 8
width = 4
calculate_rectangle(length, width)

# Second rectangle
length = 12
width = 6
calculate_rectangle(length, width)
```

Ouput:

```
PS C:\Users\SINDHU\Downloads\dev_labexam> & C:\Users\SINDHU\AppData\L
thon.exe c:/Users/SINDHU/Downloads/dev_labexam/task1.py
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

Task Description -2 (Refactoring – Modularizing Inline Calculations)

• Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Instructions:

• Identify repeated calculation patterns in the code.

• Convert them into reusable functions.

• Ensure readability and proper documentation.

Sample Code:

amount = 1000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

amount = 2000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

Expected Output:

A modular Python program that uses a reusable function to compute final amounts for different inputs.

Code:

```
def calculate_final_amount(amount):
    discount = amount * 0.10
    final_amount = amount - discount
    print("Final Amount:", final_amount)

# First amount
amount = 1000
calculate_final_amount(amount)

# Second amount
amount = 2000
calculate_final_amount(amount)
```

Output:

```
PS C:\Users\SINDHU\Downloads\dev_labexam> & C:\Users\SINDHU\AppData\Local\Programs\Python\
thon.exe c:/Users/SINDHU/Downloads/dev_labexam/task2.py
Final Amount: 900.0
Final Amount: 1800.0
```

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

• Task: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.

Instructions:

• Use AI to recommend more efficient data structures or logic

simplifications.

• Replace inefficient constructs while preserving program behavior.

• Compare execution efficiency before and after refactoring.

Sample Code:

students = ["Anil", "Bhavya", "Charan", "Divya"]

check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:

exists = False

for student in students:

if name == student:

exists = True

if exists:

print(name, "found")

else:

print(name, "not found")

Expected Output:

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

Code:

```
print("Original approach:")
start = time.time()
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
            break  # Added break for slight optimizati
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
end = time.time()
original_time = end - start
print(f"Original time: {original_time:.6f} seconds")

print("\nOptimized approach using set:")
# Optimized approach using set for O(1) lookups
students_set = set(students)
start = time.time()
```

Output:

```
Student1976 not found
Student1978 not found
Student1980 not found
Student1982 not found
Student1984 not found
Student1986 not found
Student1988 not found
Student1990 not found
Student1992 not found
Student1994 not found
Student1996 not found
Student1998 not found
Original time: 0.099569 seconds
```

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

• Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

• Detect hardcoded numeric values used directly in expressions.

• Define descriptive constants at the beginning of the program.

• Update calculations to use these constants without changing

results.

Sample Code:

print("Simple Interest:", (1000 * 2 * 5) / 100)

print("Simple Interest:", (2000 * 2 * 5) / 100)

Expected Output:

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations..

Code:

```
# Constants for simple interest calculation
RATE = 2  # Interest rate in percent
TIME = 5  # Time period in years
PERCENT = 100  # To convert percent to decimal

# Calculate simple interest for first principal
principal = 1000
simple_interest = (principal * RATE * TIME) / PERCENT
print("Simple Interest:", simple_interest)

# Calculate simple interest for second principal
principal = 2000
simple_interest = (principal * RATE * TIME) / PERCENT
print("Simple Interest:", simple_interest)
```

Output:

```
PS C:\Users\SINDHU\Downloads\dev_labexam> & C:\Users\SINDHU\AppDa
 c:/Users/SINDHU/Downloads/dev_labexam/task4.py
Simple Interest: 100.0
Simple Interest: 200.0
```

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

• Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments..

Instructions:

• Replace non-descriptive variable names with meaningful identifiers.

• Add inline comments where the logic may be unclear.

• Ensure program output remains the same.

Sample Code:

x = 50

y = 30

z = x + y

print(z)

Expected Output:

A Python program with clear variable names and explanatory comments that improve readability without altering functionality.

Code:

```
# Program to calculate the sum of two numbers

# Define the first number
first_number = 50

# Define the second number
second_number = 30

# Calculate the sum of the two numbers
sum_result = first_number + second_number

# Print the result
print(sum_result)
```

Output:

```
PS C:\Users\SINDHU\Downloads\dev_labexam> & C:\Users\SINDHU\App
 c:/Users/SINDHU/Downloads/dev_labexam/task5.py
80
```